Basic operators, maths

16 April 2026 18:40

Terms: "unary", "binary", "Operand"

- **An operand-** is what operators are applied to. For instance, in the multiplication 5 * 2, there are two operands. The left operand is 5 and the right operand is 2. Sometimes people call these arguments instead of operands.
- *An operator is unary* if it has single operand. For example, the urinary negation - reverses the sign of a number.

```
> let x = 1;
undefined
> x = -x;
-1
> console.log(x);
-1
undefined
>
```

- An operator is binary if it has two operands. The same minus exist in binary form as well:

```
> let y = 3;
undefined
> r = y - x;
4
> y
3
> x
-1
```

  We have two different operators that share the same symbol: the negation operator, a unary operator that reverses the sign, and the subtraction operator, a binary operator that subtracts one number from another.

Maths

The following math operations are supported:

- Addition "+"
- Subtraction "-"
- Multiplication "*"
- Division "/"
- Remainder "%"
- Exponentiation "**"

The first four are straightforward while "%", and "**" need a few words about them.

Remainder %

The remainder operator "%" despite its appearance, it is not related to percents.

The result of **a % b** is the remainder of the integer division of a by b.

For instance:

```
> console.log(5 % 2)
1
undefined
> console.log(8 % 3)
2
undefined
> console.log(8 % 4)
0
undefined
```

Exponentiation **

The exponentiation operator, A ** B, raises A to the power of B.

We write in our copy a^b .

For instance:

```
> let a = 5
undefined
> let b = 3
undefined
> console.log(a**b)
125
```

Just like in maths, the exponentiation operator is defined for non-integer numbers as well.

For example, a square root is an exponentiation by 1/2.

$a^{1/2}$

For instance:

```
> a = 25
25
> console.log(a**(1/2))
5
undefined
> console.log(a**(1/3))
2.924017738212866
undefined
> console.log(125**(1/3))
4.999999999999999
```

String concatenation with binary +

Let's meet the feature of Javascript operators that are beyond school arithmetic.

So usually the plus operator '+' Sums number.

But if the binary + operator is applied to string, it merges (concatenates) them:

For instance:

```
> let fn = 'Deeptanarayan'
undefined
> let ln = 'Sarkar'
undefined
> let name = fn + ' ' + ln;
undefined
> name
'Deeptanarayan Sarkar'
```

```
> console.log(name)
Deeptanarayan Sarkar
```

Note that if any of the operands is a string, then the other one is converted to a string too.
For instance:

```
> console.log('1'+ 2 + 3);
123
undefined
> console.log(1+ '2' + 3);
123
undefined
> console.log(1+ 2 + '3');
33
```

string operation → results string

bi

string operation (concatenation) with previous results.

number

3

numerical operation

```
> console.log(2 - '3');
-1
undefined
> console.log(2 / '3');
0.6666666666666666
undefined
> console.log(2 % '3');
2
```

for other arithmetic operators the string will be converted to number, if other operand or parameter is number.

```
> console.log('2' - '3')
-1
undefined
> console.log('2' / '3')
0.6666666666666666
undefined
> console.log('2' % '3')
2
```

If both string operands contains only number type, then it can perform the arithmetic operations for all the arithmetic operators. except '+' operator. Because, '+' operator in string type is used for concatenation.

Numeric conversion, unary +

The plus + exist in two forms: Binary form that we used above and the unary form.

The unary plus, or in other words the plus operator applied to a single value doesn't do anything to numbers. But if the operand is not a number, the unary plus converts it into a number.
For example:

```
> console.log(+true);
1
undefined
> y = +true
1
```

```
> console.log(y)
1
undefined
> x = +"";
0
```

It actually does the same thing as Number(...) but is shorter.
The need to convert string to number arise very often. For example, if we are getting values from HTML, form fields, they are usually strings. What if we want to sum them?

The binary plus would add them as strings:

```
> let apples = "2";
undefined
> let oranges = "3";
undefined
> console.log(apples + oranges)
23
undefined
> let totalFruits = apples + oranges;
undefined
> totalFruits
'23'
```

If we want to treat them as numbers, we need to convert and then sum them.

```
> totalFruits = +apples + +oranges;
5
```

Longer variant:

```
> console.log(Number(apples) + Number(oranges));
5
```

From a mathematician's standpoint, the abundance of pluses may seem strange. But from a programmer's standpoint, there's nothing special: unary pluses are applied first they convert string to numbers and then the binary plus sums them up.

Why are urinary pluses applied to values before the binary ones?
As we are going to see, that's because of their higher precedence.

Operator precedence:

If an expression has more than one operator. The execution order is defined by their precedence, or in other words, the default priority order of operators.

1 + 2 * 3 = 7 : The multiplication is said to have a higher precedence than an addition.

(1+2) * 5 = 15 : Parenthesis override any precedence, so if we are not satisfied with the default order, we can use them to change it.

There are many operators in Javascript. Every operator has a corresponding precedence number. The one with the larger number executes first. If the

precedence is same, the execution order is from left to right.

Precedence	Name	Sign
…	…	…
14	Unary plus	+
14	Unary negation	-
13	Exponentiation	**
12	Multiplication	*
12	Division	/
12	Modulus	%
11	Addition	+
11	Subtraction	-
…	…	…
2	Assignment	=
…	…	…

Assignment
Let's note that an assignment '=' is also an operator. It is listed in the precedence table with very low priority of 2.

That's why when we assign a variable like x = 2 * 2 + 1, The calculation are done first and then the '=' is evaluated, storing the result in x.

Assignment = returns a value

The fact of '=' being an operator, not a magical language construct, has an interesting implication.
All operators in Javascript return of value. That's obvious for '+' and '-', but also true for '='.
The call **x = value** writes the value into 'x' and then returns it.

Here is a demo that uses an assignment as a part of more complex expression:

```
> let a = 1;
undefined
> let b = 2;
undefined
> let c = 3 - (a=b+1);
undefined
> console.log(a)
3
undefined
> console.log(c)
0
```

In the example above, the result of expression (a = b + 1) is the value which was assigned to 'a' (that is 3). It is used for further evaluations.

Funny code isn't it? We should understand how it works before sometimes we see it

in Javascript libraries.
Although please don't write code like that. Such tricks definitely don't make code clearer or readable.

```
> let x, y, z;
undefined
> x = y = z = 2 + 2;
4
> x
4
> y
4
> z
4
```